

ITA0614 – MACHINE LEARNING

ASSESSMENT CO2 AT3

1. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization.

A decision tree model that achieves very high accuracy on training data but performs poorly on unseen (test) data is a clear indication of overfitting. In such cases, the model fails to generalize because it has learned the training data too precisely, including noise and irrelevant patterns, rather than capturing the true underlying relationships.

Possible Causes:

1. **Overfitting:**
The most common reason is overfitting, where the decision tree memorizes the training dataset. This results in excellent training accuracy but poor predictive performance on new data.
2. **Deep and Complex Tree Structure:**
Decision trees tend to grow until all data points are perfectly classified. A very deep tree with many nodes and branches increases model complexity and variance, making it sensitive to small changes in data.
3. **Noisy and Irrelevant Data:**
Presence of noise, outliers, or irrelevant features can mislead the model. The tree may create splits based on random fluctuations instead of meaningful patterns.
4. **Insufficient or Imbalanced Dataset:**
If the dataset is too small or imbalanced, the model may not learn general patterns properly and may bias toward dominant classes.
5. **Lack of Regularization:**
Without constraints such as depth limits or minimum samples per node, the tree grows excessively, leading to poor generalization.

Debugging Steps:

1. **Compare Training and Testing Performance:**
A large gap between training accuracy and test accuracy confirms overfitting.
2. **Use Cross-Validation:**
Apply k-fold cross-validation to evaluate the model on multiple data splits, providing a more reliable estimate of performance.
3. **Analyze Tree Structure:**
Check the depth, number of nodes, and leaf distribution. A very large tree indicates high complexity.

4. Visualize the Decision Tree:
Visualization helps identify unnecessary splits and redundant branches.
5. Inspect Feature Importance:
Determine which features contribute most to the model and identify irrelevant ones.
6. Check Data Quality:
Look for missing values, noise, and outliers that may negatively impact performance.

Optimization Methods:

1. Pruning Techniques:
 - Pre-Pruning (Early Stopping):
Set constraints such as maximum depth, minimum samples per split, or minimum samples per leaf to prevent excessive growth.
 - Post-Pruning (Cost Complexity Pruning):
Grow the full tree and then remove branches that have little impact on prediction accuracy.
2. Cross-Validation for Hyperparameter Tuning:
Use cross-validation to find optimal values for parameters like depth, split criteria, and minimum samples, ensuring a balance between bias and variance.
3. Feature Selection:
Remove irrelevant or redundant features using methods such as correlation analysis, feature importance ranking, or recursive feature elimination. This reduces noise and improves model clarity.
4. Handling Noisy Data:
Apply data preprocessing techniques such as outlier removal, normalization, and data cleaning to improve data quality.
5. Ensemble Methods (Optional Improvement):
Techniques like Random Forest or Gradient Boosting combine multiple trees to reduce overfitting and improve generalization.
6. Regularization Parameters:
Adjust parameters like maximum leaf nodes, minimum impurity decrease, and splitting criteria to control model complexity.

Conclusion:

The poor performance of the decision tree on unseen data is mainly due to overfitting caused by excessive complexity and noisy data. By applying systematic debugging steps and optimization techniques such as pruning, cross-validation, and feature selection, the model can be simplified and made more robust. This improves its ability to generalize and deliver accurate predictions on new data, achieving a better balance between bias and variance.

2. A convolutional neural network (CNN) used for image classification produces unstable results across training epochs. Examine potential issues such as improper data augmentation, vanishing gradients, or batch normalization errors. Describe debugging strategies and suggest optimization techniques to stabilize training and improve accuracy.

A Convolutional Neural Network (CNN) used for image classification showing unstable results across training epochs indicates issues in the training process, optimization, or data handling. This instability may appear as fluctuating accuracy/loss, failure to converge, or inconsistent validation performance.

Possible Causes:

1. **Improper Data Augmentation:**
Excessive or unrealistic transformations (e.g., extreme rotations, flips, or distortions) can distort the data distribution, making it difficult for the model to learn meaningful features. Inconsistent augmentation between training and validation sets can also lead to instability.
2. **Vanishing/Exploding Gradients:**
In deep CNNs, gradients may become extremely small (vanishing) or very large (exploding), preventing effective weight updates. This leads to slow learning or unstable oscillations during training.
3. **Batch Normalization Errors:**
Incorrect placement of batch normalization layers or improper handling of training vs inference mode can cause fluctuations. Small batch sizes can also lead to inaccurate batch statistics.
4. **Learning Rate Issues:**
A very high learning rate may cause the model to overshoot the optimal solution, leading to oscillations. A very low learning rate may slow convergence and produce unstable patterns.
5. **Weight Initialization Problems:**
Poor initialization can result in uneven gradient flow, affecting convergence.
6. **Overfitting or Underfitting:**
Lack of regularization or insufficient model capacity may also cause unstable validation results.

Debugging Strategies:

1. **Monitor Training and Validation Curves:**
Plot loss and accuracy across epochs to identify patterns such as oscillations, divergence, or overfitting.
2. **Check Data Pipeline:**
Verify preprocessing steps, normalization, and augmentation consistency. Ensure validation data is not augmented excessively.

3. **Gradient Inspection:**
Track gradient values to detect vanishing or exploding gradients.
4. **Validate Batch Normalization Behavior:**
Ensure correct switching between training and evaluation modes and check batch size adequacy.
5. **Experiment with Learning Rate:**
Try different learning rates and observe their effect on convergence.
6. **Run with Small Subset:**
Train on a small dataset to check if the model can overfit easily; failure indicates implementation issues.

Optimization Techniques:

1. **Learning Rate Tuning:**
Use learning rate scheduling (step decay, cosine annealing) or adaptive optimizers like Adam to stabilize updates.
2. **Proper Data Augmentation:**
Apply realistic and moderate augmentations. Techniques like random crop, horizontal flip, and normalization should preserve class features.
3. **Gradient Stabilization:**
Use techniques such as gradient clipping to prevent exploding gradients and proper activation functions (e.g., ReLU) to reduce vanishing gradients.
4. **Batch Normalization Optimization:**
Place batch normalization after convolution layers and before activation functions. Use sufficiently large batch sizes or alternatives like layer normalization if needed.
5. **Weight Initialization:**
Use appropriate initialization methods such as He or Xavier initialization for better gradient flow.
6. **Regularization Techniques:**
Apply dropout, L2 regularization, and early stopping to improve generalization and reduce fluctuations.
7. **Architecture Improvements:**
Use residual connections (ResNet) to improve gradient flow in deep networks.

Conclusion:

Unstable CNN training is typically caused by issues in data preprocessing, gradient flow, or hyperparameter settings. By carefully debugging the data pipeline, monitoring training behavior, and applying optimization techniques such as learning rate tuning, batch normalization, and proper augmentation, the model can achieve stable convergence and improved accuracy on image classification tasks.